Lower Bound Theory Concept

Module 3

Lower Bound Theory Concept

- Based upon the calculation of minimum time that is required to execute an algorithm is known as a **lower bound theory or Base Bound Theory.**
- Aim: The main aim is to calculate a minimum number of comparisons required to execute an algorithm.
- Lower Bound Theory uses a number of methods/techniques to find out the lower bound.

- According to the lower bound theory , for lower bound L(n) of an algorithm it is not possible to have any any algorithm(for a common problem) whose time complexity is less than L(n)for random input
- Once lower bound is calculated ,then we can compare it with the actual complexity of the algorithm and if their order are same then we can declare our algorithm as optimal

Techniques:

- The techniques which are used by lower Bound Theory are:

1. Comparisons Trees.
2. Oracle and adversary argument
3. State Space Method

Comparisons Trees.

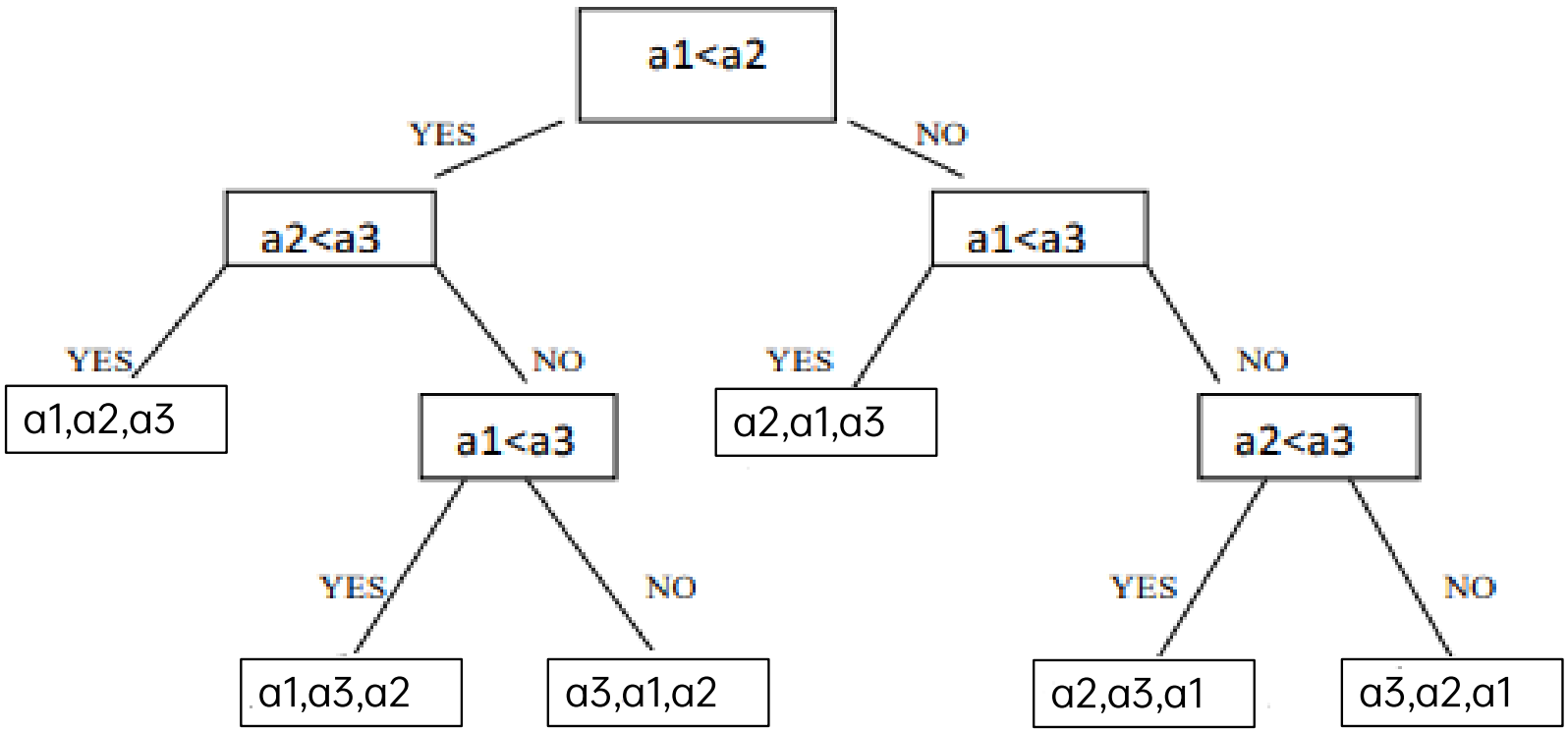
- Are the computational model useful for determining lower bounds for searching and sorting problems.

- The Comparison based algorithm defines a decision tree.

Decision Tree

- A decision tree is a full binary tree that shows the comparisons between elements that are executed by an appropriate sorting algorithm operating on an input of a given size.
- Control, data movement, and all other conditions of the algorithm are ignored.

- In a decision tree, there will be an array of length n.
- So, total leaves will be n! (I.e. total number of comparisons)
- Taking an Example of comparing a1, a2, and a3.
- Left subtree will be true condition i.e. ai ≤ aj
- Right subtree will be false condition i.e. ai >aj


a1<a2
YES
NO
a2<a3
a1<a3
YES
NO
YES
NO
a1,a2,a3
a1<a3
a2,a1,a3
a2<a3
YES
NO
YES
NO
a1,a3,a2
a3,a1,a2
a2,a3,a1
a3,a2,a1

Lower bound for comparison based sorting algorithms

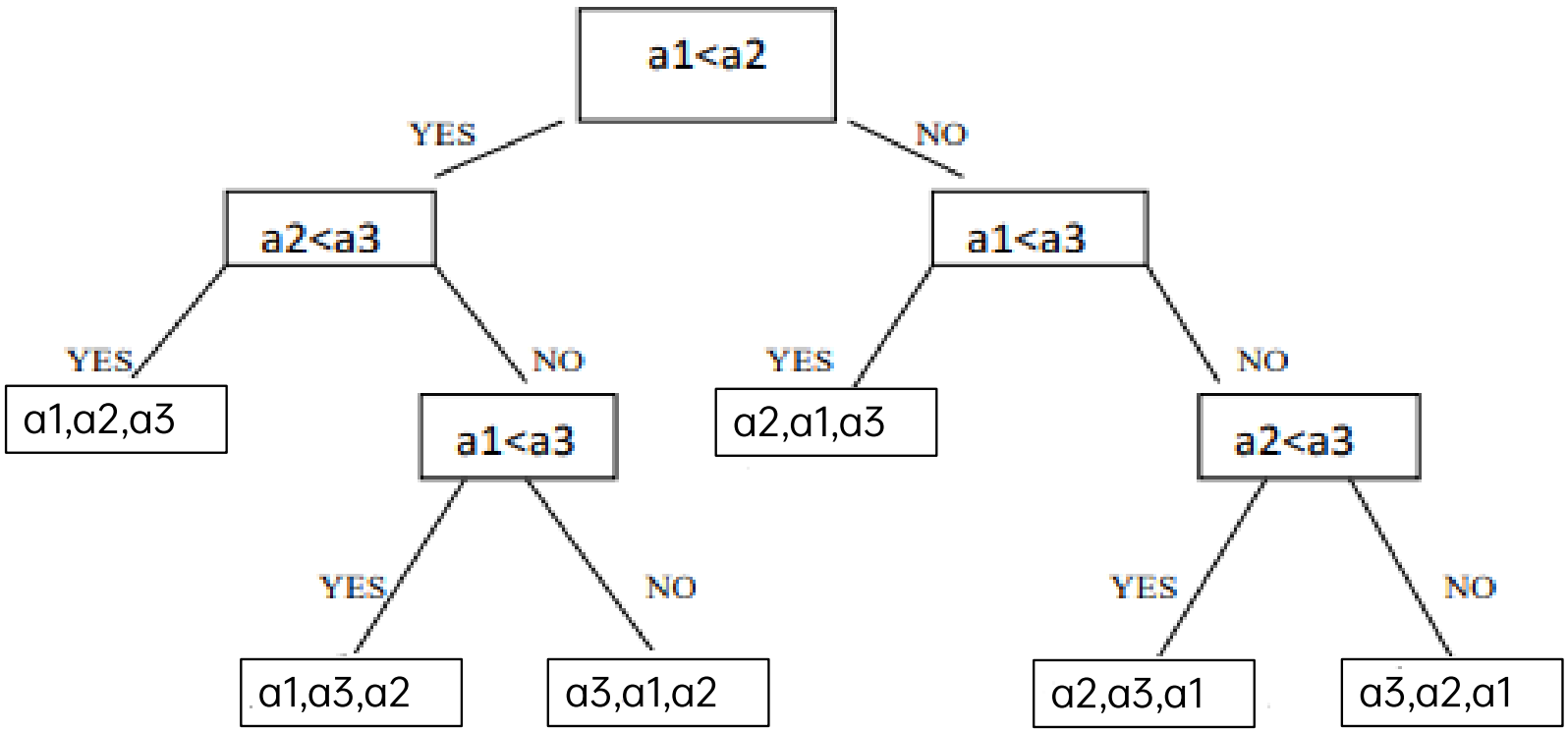
- A sorting algorithm **is comparison based** if it uses comparison operators to find the order between two numbers.
- Comparison sorts can be viewed abstractly in terms of decision trees.
- A decision tree is a full binary tree that represents the comparisons between elements that are performed by a particular sorting algorithm operating on an input of a given size.
- The execution of the sorting algorithm corresponds to tracing a path from the root of the decision tree to a leaf.
- At each internal node, a comparison $a_i <= a_j$ is made.
- The left subtree then dictates subsequent comparisons for $a_i <= a_j$, and the right subtree dictates subsequent comparisons for $a_i > a_j$.
- When we come to a leaf, the sorting algorithm has established the ordering.
- So we can say following about the decision tree.

- The length of the longest path from the root of the decision tree to a leaf represents the worst case number of comparisons that the sorting algorithm performs.

- Therefore, worst-case number of comparisons is the height of the decision tree.

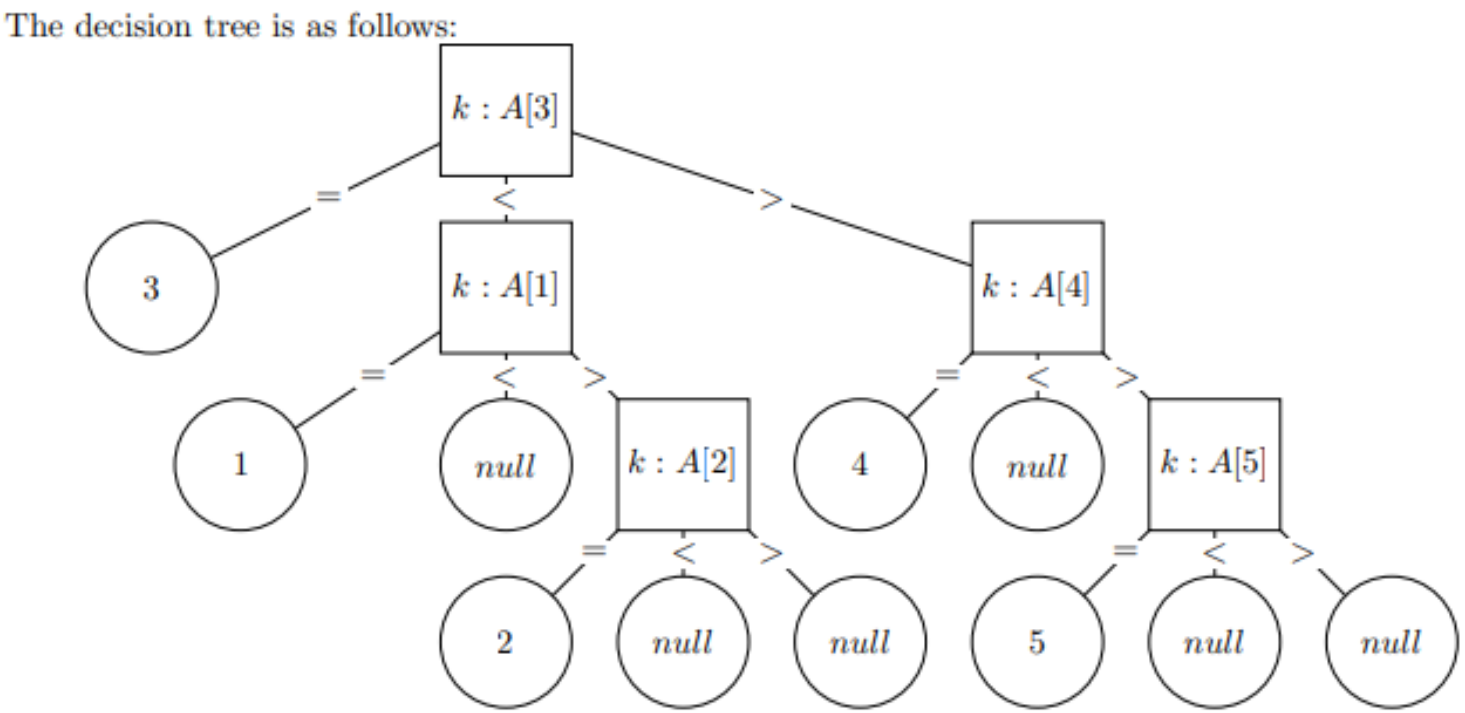
- How can we find the algorithm with the smallest worst-case number of comparisons?
  - We find the decision tree with the smallest height.

- In other words, to find the worst-case running time of the ”best” algorithm (i.e., the one with the smallest worst-case number of comparisons), we want to find a lower bound on the height of the decision trees.


a1<a2
YES
NO
a2<a3
a1<a3
YES
NO
YES
NO
a1,a2,a3
a1<a3
a2,a1,a3
a2<a3
YES
NO
YES
NO
a1,a3,a2
a3,a1,a2
a2,a3,a1
a3,a2,a1

Lower bounds for comparison based search

- Searching a sorted array



1. Some leaves are duplicates (i.e. the ones that return null).
2. There are n + 1 unique leaves (we can return each of the 5 keys, and null if the value is not in the array).
3. Each non-leaf node has three children.